

Resenha Crítica: Artigo 7 — Domain-Driven Design Reference

Gustavo Pessoa Firmino Duarte

12 de Abril de 2026

1 Introdução

O *Domain-Driven Design Reference* (2015) de Eric Evans é um guia de referência conciso que sintetiza os padrões e conceitos fundamentais do *Domain-Driven Design* (DDD), abordagem que Evans introduziu em seu livro homônimo de 2003. O documento não pretende substituir o livro original, mas oferecer uma referência rápida e precisa para praticantes que já conhecem o paradigma. Seu valor está em tornar explícitos os padrões que o DDD defende: desde a construção de uma *linguagem ubíqua* compartilhada entre desenvolvedores e especialistas do domínio, passando pelo isolamento do modelo em *contextos delimitados* (*bounded contexts*), até os blocos construtivos táticos como entidades, objetos de valor, agregações, repositórios e serviços de domínio.

2 Linguagem Ubíqua, Contextos Delimitados e Blocos Táticos

O conceito mais transformador do DDD é a *linguagem ubíqua* (*ubiquitous language*): uma linguagem comum, rigorosa e sem ambiguidade, construída colaborativamente por desenvolvedores e especialistas do negócio, que deve permear o código, a documentação e as discussões da equipe. Evans argumenta que a separação entre a “linguagem dos negócios” e a “linguagem do código” é uma fonte constante de erros e mal-entendidos; quando o código usa os mesmos termos que os especialistas usam no dia a dia, a descoberta de inconsistências no modelo de domínio torna-se mais fácil.

Os *bounded contexts* definem as fronteiras dentro das quais um modelo de domínio é válido e consistente. Em sistemas grandes, diferentes sub-domínios possuem modelos distintos e frequentemente conflitantes para o mesmo conceito — um “cliente” em um contexto de faturamento pode ter atributos completamente diferentes de um “cliente” em um contexto de suporte. O DDD abraça essa multiplicidade, desde que as fronteiras sejam explícitas e as traduções entre contextos sejam gerenciadas através de padrões como *Anti-Corruption Layer*, *Shared Kernel* ou *Open Host Service*.

No nível tático, Evans distingue **entidades** (objetos com identidade ao longo do tempo) de **objetos de valor** (*value objects*, imutáveis e definidos apenas por seus atributos). As **agregações** (*aggregates*) são clusters de entidades e objetos de valor tratados como uma unidade transacional, acessados exclusivamente através de uma raiz de agregação. **Repositórios** abstraem o mecanismo de persistência, e **eventos de domínio**

(*domain events*) capturam acontecimentos significativos no domínio, viabilizando integração assíncrona entre contextos.

3 Conexão com a Prática Profissional

Ao desenvolver o MVP de e-commerce com Node.js, Prisma ORM e React, implementei estruturas que, em retrospecto, guardam semelhança com os padrões táticos do DDD — ainda que sem o rigor conceitual que Evans propugna. Os modelos do Prisma atuavam como entidades, e usei serviços para encapsular a lógica de negócio. Porém, a ausência de agregações bem definidas resultou em operações que modificavam múltiplas entidades relacionadas fora de transações atômicas, criando janelas de inconsistência de dados.

Na ArcelorMittal, a diversidade de sistemas internacionais que suportei ilustrou claramente a problemática dos contextos delimitados: o mesmo conceito de “ordem” (*order*) tinha semânticas distintas no sistema de produção brasileiro, no sistema logístico europeu e no sistema financeiro global. A ausência de *bounded contexts* explícitos tornava as integrações entre esses sistemas frágeis, com regras de tradução espalhadas e frequentemente duplicadas.

A linguagem ubíqua é um aspecto que percebo como especialmente difícil de cultivar: requer disciplina contínua para que termos do domínio sejam usados com consistência no código, nos tickets e nas discussões — uma prática que raramente vi implementada de forma sistemática em projetos reais.

4 Conclusão

O *Domain-Driven Design Reference* de Evans é um compendio essencial para qualquer desenvolvedor que trabalhe com sistemas complexos. Seus padrões — especialmente bounded contexts, agregações e linguagem ubíqua — oferecem ferramentas concretas para domar a complexidade accidental que frequentemente contamina o modelo de domínio em sistemas de longa vida. O DDD não é uma bala de prata: demanda investimento em compreensão do negócio e disciplina de design. Mas para sistemas com domínios ricos e equipes colaborativas, os benefícios em clareza, manutenibilidade e agilidade diante de mudanças de negócio são substanciais.